

1944. Number of Visible People in a Queue

Solved 

Hard Topics Companies Hint

There are n people standing in a queue, and they numbered from 0 to $n - 1$ in left to right order. You are given an array `heights` of distinct integers where `heights[i]` represents the height of the i^{th} person.

A person can see another person to their right in the queue if everybody in between is shorter than both of them. More formally, the i^{th} person can see the j^{th} person if $i < j$ and $\min(\text{heights}[i], \text{heights}[j]) > \max(\text{heights}[i+1], \text{heights}[i+2], \dots, \text{heights}[j-1])$.

Return an array `answer` of length n where `answer[i]` is the number of people the i^{th} person can see to their right in the queue.

Example 1:



Input: `heights = [10,6,8,5,11,9]`

Output: `[3,1,2,1,1,0]`

Explanation:

Person 0 can see person 1, 2, and 4.

Person 1 can see person 2.

Person 2 can see person 3 and 4.

Person 3 can see person 4.

Person 4 can see person 5.

Person 5 can see no one since nobody is to the right of them.

Example 2:

Input: `heights = [5,1,2,3,10]`

Output: `[4,1,1,1,0]`

Constraints:

- $n == \text{heights.length}$
- $1 \leq n \leq 10^5$
- $1 \leq \text{heights}[i] \leq 10^5$
- All the values of `heights` are unique.

```
class Solution {
    public int[] canSeePersonsCount(int[] heights) {
        Deque<Integer> stackMono = new ArrayDeque<>();
        int[] result = new int[heights.length];
        for (int i = heights.length - 1; i >= 0; i--) {
            int count = 0;
            while (!stackMono.isEmpty() && heights[i] > stackMono.peek()) {
                count++;
            }
            stackMono.add(heights[i]);
            result[i] = count;
        }
        return result;
    }
}
```

```
        stackMono.pop();
    }
    if (!stackMono.isEmpty()) {
        count++;
    }
    result[i] = count;
    stackMono.push(heights[i]);
}
return result;
}
}
```

Right to left:

$O(N)$

→ Traverse from right to left.

→ for every index from right to left:

* While stack is not empty and stack top is $<$ current element, it means we can see that element, so pop and count++;

* for every index, we can see the next highest index, so we can surely see the stack top. So if stack is not empty then count++.

* $result[i] = count$ & push element to stack.

→ Return result.

* The stack we use is strictly decreasing because if two elements are of same height, we can see the buildings beyond the second one.